

MODULE 6
IVI: In-Vehicle Infotainment Systems

**Conceptual Architecture Diagram of an IVI System Integrating
Media, Navigation and Projection Features**

with Annotated Component Interfaces and Data Flow

Submitted by

Name: Rohit Roy

Roll No.: 41

Enrolment No.:12023002002115

Wipro Automotive Course

Electronics and Communication Engineering

Institute Of Engineering and Management, Newtown

Under the guidance of

Subindu Saha

Date of submission: 29.09.2023

2. Abstract

This report presents the conceptual architecture of an In-Vehicle Infotainment (IVI) system that integrates media playback, navigation and smartphone projection (Android Auto and Apple CarPlay). The design follows the layered structure used by Android-based automotive platforms: driver-facing apps, framework services, a car service layer, vendor Hardware Abstraction Layers (HALs) and the physical hardware.

Thirteen component-to-component interfaces are identified and annotated with the protocol or API they use and the data they carry. Three end-to-end data flows (media playback, navigation and projection) are traced step by step using twelve numbered markers on the diagram. The design is verified with a requirement-coverage checklist and a walkthrough of a combined scenario in which a navigation voice prompt interrupts music.

The outcome is a clear reference model that explains how user input travels down through the stack to the hardware, and how sensor and vehicle data travel back up to the driver. The work is conceptual: it is a design study and no real vehicle bus, hardware or source code was used.

Keywords: IVI, infotainment, Android Automotive, HAL, Vehicle HAL, audio focus, Android Auto, Apple CarPlay, system architecture.

3. Introduction

3.1 What an IVI system is and why it matters

An In-Vehicle Infotainment (IVI) system is the combined hardware and software unit in the dashboard, usually called the head unit, that provides information and entertainment to the people in a vehicle. Typical functions are media playback (radio, USB, streaming), navigation, hands-free calling, smartphone integration, voice control and access to some vehicle settings. Modern IVI systems are software-defined computers with a touchscreen, microphones, speakers and connections to the rest of the vehicle.

IVI matters for three main reasons:

- **Driver experience.** The head unit is the screen the driver uses most. A consistent interface, fast response, voice control and familiar phone apps directly affect how the car feels to use and are a major buying factor.
- **Safety.** A poorly designed system distracts the driver. Good IVI design supports hands-free operation, limits complex tasks while driving, gives clear navigation prompts and can show warnings and the rear-view camera image.
- **Connectivity.** IVI connects the car to the phone (USB, Bluetooth, Wi-Fi), to the internet and to cloud services, and allows software updates over the air (OTA).

3.2 Objective of the chosen option

Option 1 was chosen: “design a conceptual architecture diagram of an IVI system integrating media, navigation and projection features, and annotate component interfaces and data flow.” This option was selected because it can be completed fully and accurately as a design study without special hardware, and it builds the foundation needed to understand the other three options.

The specific objectives are:

- To identify the main components of an IVI system needed for media, navigation and projection.
- To arrange these components in a clear layered architecture.
- To annotate every interface between components with the protocol, API or bus used and the data exchanged.
- To annotate the data flow by tracing how a request or a piece of data moves through the stack for each of the three features.
- To verify that the design meets the task requirements.

3.3 Scope and limitations

- The work is **conceptual and simulation-free**: it is a design and modelling study. No real vehicle, real CAN bus, head-unit hardware or production source code was used.
- The architecture is based on publicly documented **Android Automotive OS concepts** (Car service, Vehicle HAL, audio focus and so on). Real products from different manufacturers vary in detail.
- Some internal parts are deliberately simplified to keep the diagram readable (see Section 9.2).
- Topics such as Bluetooth telephony, rear-view camera, instrument cluster, cybersecurity and OTA updates are outside the diagram and are listed as future scope.
- Performance figures (latency, memory, boot time) are not measured because nothing was executed.

4. Background / Literature

4.1 Evolution of IVI platforms

Early car radios were closed devices with fixed functions. Later infotainment units used proprietary embedded software that was different for every manufacturer and difficult to update. Today most head units run a general-purpose operating system, which allows richer apps, standard development tools and regular updates. The main platforms and approaches in use are:

Platform / approach	Short description
Android Automotive OS (AAOS)	A full Android-based operating system that runs directly on the car's head unit. It includes automotive-specific parts such as Car service and Vehicle HAL. Apps for media and navigation are installed on the car itself.
Android Auto	A phone-projection technology. Apps run on the smartphone and the phone sends a user interface to the car screen over USB or wireless. The car provides display, speakers and controls.
Apple CarPlay	Apple's phone-projection technology for iPhone, with the same basic idea as Android Auto: the phone runs the apps and the car acts as display and input device.
QNX and Automotive Grade Linux (AGL)	Other operating systems used in head units. QNX is a real-time operating system from BlackBerry. AGL is an open-source Linux-based platform from the Linux Foundation.

Because Android is open source and widely known by developers, this report uses the Android automotive architecture as its reference model.

4.2 Layered Android automotive architecture and interfaces

Android-based automotive systems are organised in layers so that apps do not depend on specific hardware. The key ideas used in this report are summarised below.

Concept	Meaning
Apps layer	Applications the user sees: media, navigation and projection user interfaces.
Framework services	System services that apps call through Binder IPC, for example the media session service and the location service.
Car service	Automotive-specific system service. It includes Car audio service (audio focus, volume) and Car property service (access to vehicle signals such as speed and gear).
Binder IPC and AIDL	Binder is Android's inter-process communication mechanism. AIDL (Android Interface Definition Language) is used to define the interfaces that run over Binder.
Hardware Abstraction Layer (HAL)	A standard interface between Android and vendor-specific hardware drivers. Examples: Audio HAL, GNSS HAL and Vehicle HAL. HALs are defined in AIDL (older releases used HIDL).
Vehicle HAL (VHAL)	The HAL that exposes vehicle data and controls as named properties (for example vehicle speed and gear selection) that can be read, written or subscribed to.
Audio focus and audio contexts	A rule system that decides which sound source may play when several want to play at once. Automotive audio uses contexts such as music, navigation, voice command, call and alarm, so navigation prompts can be treated differently from music.
Projection	Phone-based apps (Android Auto, CarPlay) send video and audio to the head unit, and the head unit sends touch and other input back. Connection is by USB, or wirelessly using Bluetooth for setup and Wi-Fi for data.

Concept	Meaning
Vehicle networks	ECUs (Electronic Control Units) in the car communicate on networks such as CAN. The head unit normally reaches them through a gateway or microcontroller.

5. Tools & Environment

Option 1 is a design task, so no special hardware or runtime environment is needed. The tools used are listed below.

Tool	Purpose	Notes
Microsoft Word (or LibreOffice Writer)	Writing and formatting this report	Any word processor can be used
Diagram tool	Drawing the architecture diagram (Figure 1)	Figure 1 in this report was produced with Python and the Matplotlib library. It can be redrawn with draw.io (diagrams.net), PowerPoint or Lucidchart (Appendix C)
Android Open Source Project (AOSP) documentation	Checking correct names of components and interfaces	Automotive section of source.android.com
Android for Cars developer documentation	Media, navigation and projection concepts	developer.android.com/cars

Environment. A normal laptop or desktop with a web browser is sufficient. No Android Studio, emulator or Raspberry Pi is required for this option; those are needed only for Option 2.

6. System Design / Methodology

6.1 Design approach

The design was developed top-down. First the required user functions were listed, then the components needed for each function were identified and placed in layers, and finally the connections between the components were defined and annotated.

Required function	Components chosen
Media	Media app, Media service, Car audio service, Audio HAL, speakers and amplifier
Navigation	Navigation app, Location service, GNSS HAL, GPS receiver (plus Car audio service and Audio HAL for voice guidance)
Projection	Projection app, Projection service, smartphone (external device), Car property service and Vehicle HAL for driving-state data

6.2 Layered design and design rules

The system is arranged in six layers, from the driver at the top to the hardware at the bottom.

Layer	Components	Responsibility
Driver	Driver or passenger, touchscreen, microphone, steering-wheel keys	Gives input and receives visual and audio output
Apps	Media app, Navigation app, Projection app	Present the user interface and implement the use cases
Services	Media service, Location service, Projection service	Framework services that manage playback sessions, location and the phone session
Car layer	Car audio service, Car property service	Automotive-specific services for audio focus and volume, and for vehicle signals
HAL	Audio HAL, GNSS HAL, Vehicle HAL	Standard vendor-implemented interfaces that hide hardware details
Hardware	Speakers and amplifier, GPS receiver, CAN bus and ECUs (smartphone is an external device)	Physical devices

The following design rules were applied while drawing the connections:

- Apps communicate only with framework services through Binder/AIDL. They never talk to a HAL or to hardware directly.
- Framework and car services reach hardware only through HAL interfaces.
- Vendor-specific code exists only in the HAL and driver layers, so apps stay hardware independent.
- Vehicle signals reach apps only through Car property service, which allows permissions and safety rules to be enforced.
- Audio focus is decided in one place, Car audio service, so media, navigation prompts and calls do not conflict.

6.3 Conceptual architecture diagram

Figure 1 shows the complete architecture. Solid arrows with two heads show that data or commands move in both directions across an interface. The text on the right side of the figure names the interface used between each pair of layers. The coloured circles are data-flow markers that are explained in Section 7.3.

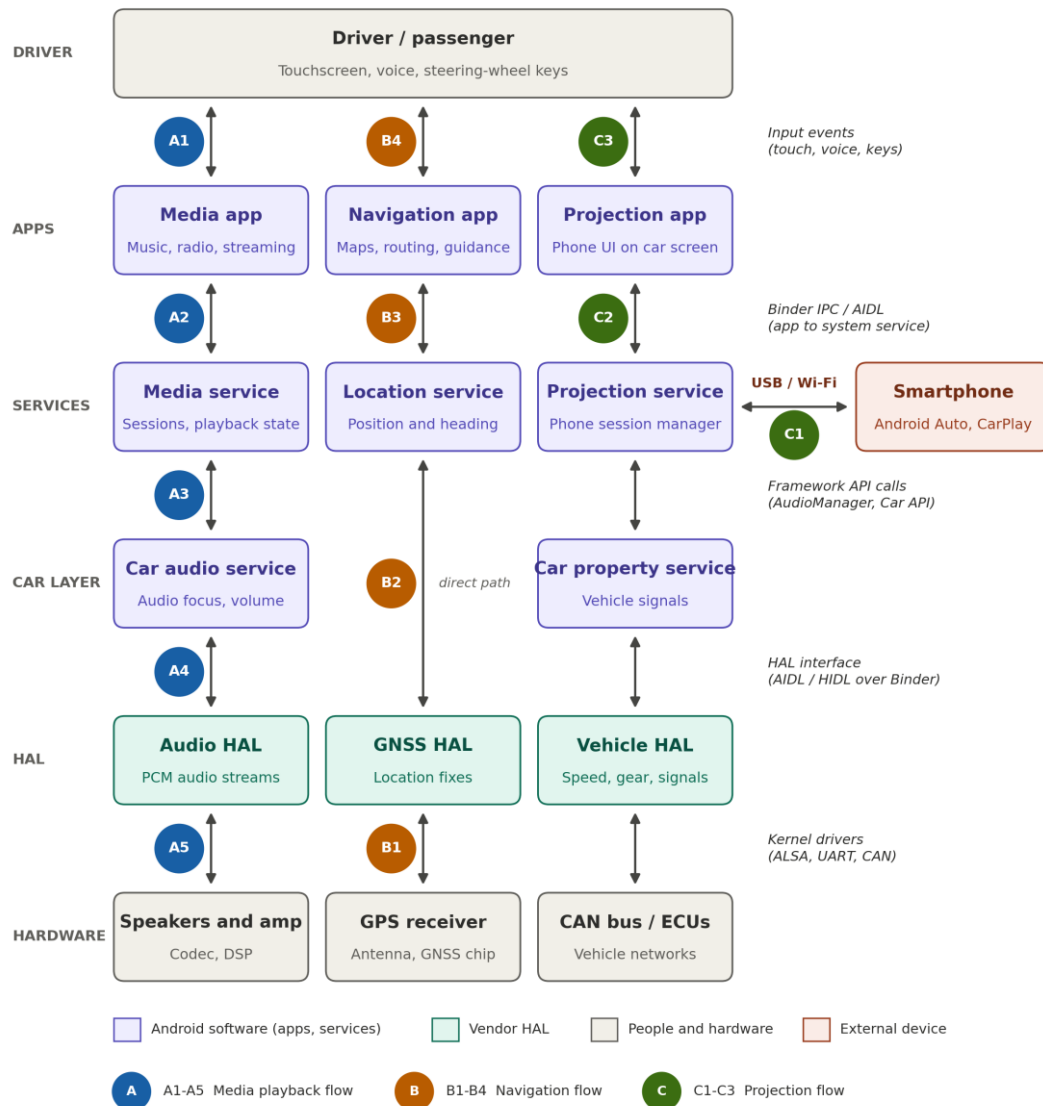


Figure 1: Conceptual architecture of an IVI system integrating media, navigation and projection, with annotated interfaces and data flow

Draw.io – Diagram :

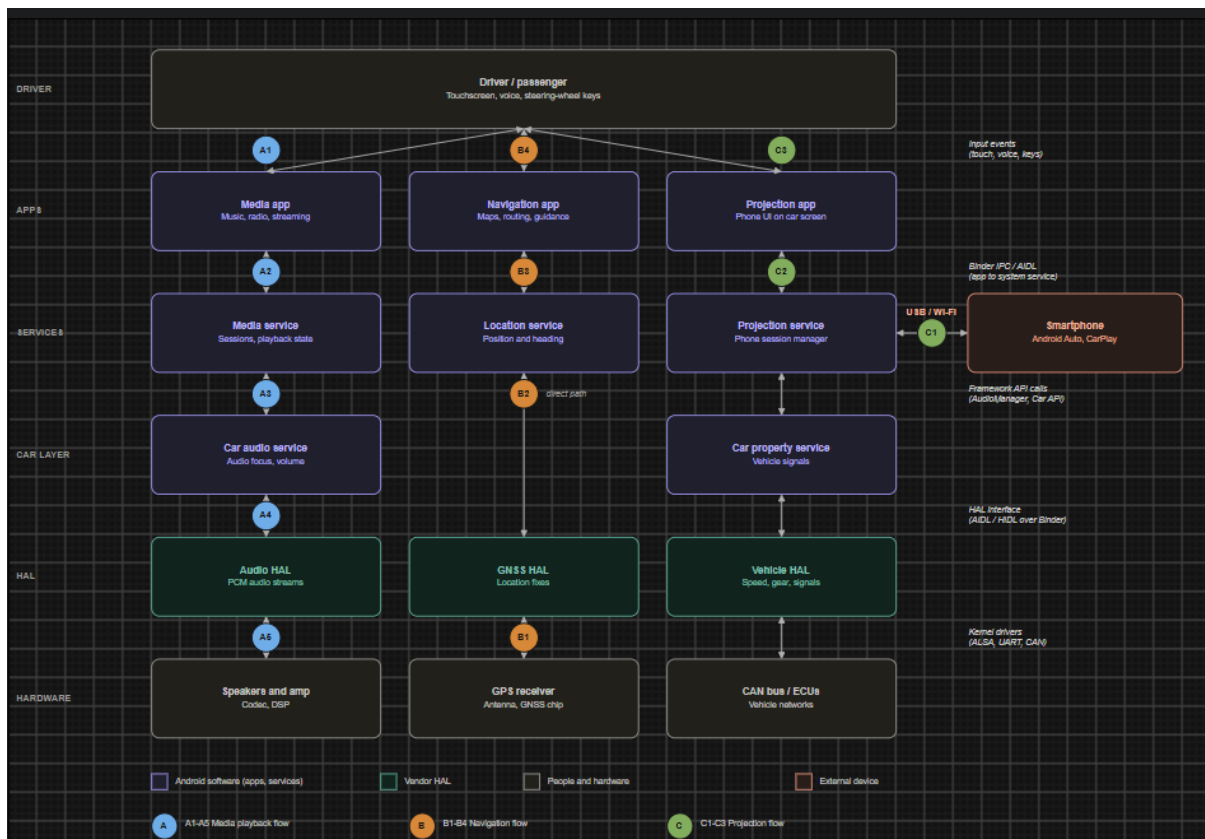


Fig 2 : Diagram of Handmade Picture from Draw.io

6.4 Annotate component interfaces and data flow:

The task asks for two kinds of annotation on the diagram.

- **Annotating component interfaces** means labelling each connection between two components with **how** they talk to each other: the protocol, API or bus used (for example Binder IPC/AIDL, HAL interface, USB, CAN). In Figure 1 this is done with the italic labels on the right and in the interface table in Section 7.2.
- **Annotating data flow** means showing **what** data moves along the connections and **in what order** for a given use case. In Figure 1 this is done with the numbered markers A1 to A5, B1 to B4 and C1 to C3, and in the step tables in Section 7.3.

In short: an interface annotation answers "how do they communicate", and a data-flow annotation answers "what travels, and in which direction and order".

6.5 Design assumptions

- The reference platform is an Android-based head unit that follows Android Automotive concepts. Names such as Car service and Vehicle HAL are taken from the Android Open Source Project.

- Projection is modelled as a generic projection app and projection service pair, with the smartphone as an external device. The exact integration of Android Auto or CarPlay differs between manufacturers.
- Location and GNSS use the standard Android path (Location service to GNSS HAL). For this reason the diagram shows a direct path that does not pass through the Car layer.
- Every arrow is shown as bidirectional because each interface carries requests one way and responses or status the other way. The direction of the main data is given in the step tables.

7. Implementation / Modelling

7.1 Modelling procedure

The diagram was modelled in the following steps:

1. List the required functions from the task: media, navigation and projection.
2. Identify the components for each function and place them in layers (Section 6.2).
3. Connect neighbouring components with arrows, following the design rules.
4. Annotate every interface with its protocol or API and the data it carries (Section 7.2).
5. Trace one end-to-end data flow for each function and number every step (Section 7.3).
6. Trace a combined scenario to check how the functions interact (Section 7.4).
7. Verify the result against the requirements and record simplifications (Sections 8 and 9).

7.2 Component interface annotations

Table 1 lists all thirteen interfaces in Figure 1. The first column is the identifier used in this report.

ID	Between	Interface / protocol	Data exchanged
I1	Driver and Apps (Media, Navigation, Projection)	Input events: touchscreen, steering keys and microphone through the Linux input subsystem and the Android input framework	Touch and key events, voice commands; screen and sound output back to the driver
I2	Media app and Media service	Binder IPC (MediaSession and MediaBrowser APIs)	Play, pause and skip commands; track metadata; playback state
I3	Navigation app and Location service	Binder IPC (LocationManager API)	Location update requests; location updates
I4	Projection app and Projection service	Binder IPC (car projection APIs)	Video and audio stream, touch events, session state
I5	Media service and Car audio service	Framework API calls (AudioManager and Car API)	Audio focus requests; focus granted or lost; volume

ID	Between	Interface / protocol	Data exchanged
I6	Projection service and Car property service	Binder IPC (Car API)	Driving state, speed, gear (used to limit distracting features while driving)
I7	Car audio service and Audio HAL	HAL interface (AIDL or HIDL over Binder), through the native audio server	Audio routing, volume, PCM audio stream
I8	Location service and GNSS HAL	HAL interface (AIDL or HIDL)	Location fixes, satellite status, configuration
I9	Car property service and Vehicle HAL	Vehicle HAL interface (AIDL; HIDL in older releases)	Get, set and subscribe to vehicle properties such as speed and gear
I10	Audio HAL and Speakers / amplifier	Kernel audio driver (ALSA / TinyALSA); digital audio bus such as I2S or TDM to codec, DSP and amplifier	Digital audio samples and control settings
I11	GNSS HAL and GPS receiver	Driver over UART or SPI, or an integrated receiver in the SoC	NMEA or vendor-format position data
I12	Vehicle HAL and CAN bus / ECUs	Vehicle network, normally through a gateway or microcontroller (CAN frames)	Vehicle signals and status
I13	Projection service and Smartphone	USB, or Wi-Fi with Bluetooth for wireless setup	Video, audio, touch input, vehicle data

Table 1: Interface annotations for Figure 1

7.3 Data-flow annotations

Three end-to-end flows are traced. Each step number matches a marker in Figure 1: **A** (blue) is media playback, **B** (orange) is navigation and **C** (green) is projection.

Flow A: Media playback (user input propagates down to the speakers)

Step	From to	Data	Interface
A1	Driver to Media app	Touch on the Play button	I1 Input events
A2	Media app to Media service	Play command; track metadata and playback state return upward	I2 Binder IPC
A3	Media service to Car audio service	Audio focus request (music context); focus granted or denied	I5 Framework API calls
A4	Car audio service to Audio HAL	Routing and volume decision; PCM audio stream	I7 HAL interface
A5	Audio HAL to Speakers and amplifier	Digital audio samples converted to sound	I10 Driver and audio bus

Flow B: Navigation (position data propagates up to the driver)

Step	From to	Data	Interface
B1	GPS receiver to GNSS HAL	Position data computed from satellite signals	I11 UART, SPI or SoC
B2	GNSS HAL to Location service	Location fix: latitude, longitude, speed, bearing, accuracy, time	I8 HAL interface
B3	Location service to Navigation app	Location updates for the registered listener	I3 Binder IPC
B4	Navigation app to Driver	Map and route on screen; voice guidance is sent through the audio path A3 to A5 with a navigation audio context	I1 output; I5, I7, I10 for voice

Flow C: Projection (phone content shown on the car display)

Step	From to	Data	Interface
C1	Smartphone to Projection service	Video, audio and session data	I13 USB or Wi-Fi
C2	Projection service to Projection app	Video and audio stream	I4 Binder IPC
C3	Projection app to Driver	Phone user interface on the car display. Touch events return along C3, C2, C1 to the phone. Driving state from I6 and I9 limits distracting features.	I1 Input and display

Vehicle-data path (not numbered). Vehicle signals travel from the CAN bus and ECUs (I12) through Vehicle HAL (I9) to Car property service and then to the Projection service (I6) or other apps, so that the user interface can react to driving state.

7.4 Combined scenario: navigation prompt during music

This scenario shows how the three flows interact through audio focus.

1. Music is playing through the path A1 to A5.
2. The navigation app receives a location update (B3) and needs to speak a turn instruction.
3. The navigation app requests audio focus with a navigation audio context (B4 with I5).
4. Car audio service applies the focus rules. In a typical configuration the music is lowered in volume (ducked) while the prompt plays. The exact behaviour is set by the manufacturer.
5. Car audio service sets routing and volume for the Audio HAL (I7) and the prompt is played through the speakers (I10).

6. When the prompt ends, the navigation app abandons focus and the music volume is restored.

8. Testing & Results

Because this is a design study, testing is done by **design verification**: a requirement-coverage check, traceability of the data flows and consistency checks. No code was executed.

8.1 Requirement coverage

ID	Requirement	Where satisfied	Result
R1	Architecture integrates media	Media app, Media service, Car audio service, Audio HAL, speakers (Figure 1, Flow A)	Met
R2	Architecture integrates navigation	Navigation app, Location service, GNSS HAL, GPS receiver (Figure 1, Flow B)	Met
R3	Architecture integrates projection	Projection app, Projection service, smartphone (Figure 1, Flow C)	Met
R4	Component interfaces are annotated	13 interfaces I1 to I13 (Table 1 and right side of Figure 1)	Met
R5	Data flow is annotated	12 numbered markers A1 to A5, B1 to B4, C1 to C3 (Section 7.3)	Met

8.2 Consistency checks

ID	Check	Method	Result
K1	Every arrow in Figure 1 corresponds to an interface in Table 1 (15 arrows, 13 interfaces, because I1 covers 3 arrows and I10 to I12 cover the hardware arrows)	Manual count	Pass
K2	No app is connected directly to a HAL or to hardware	Visual inspection	Pass
K3	Every numbered marker maps to one row in a step table	Cross-reference	Pass
K4	Flows are traceable from the source to the destination without a gap	Walkthrough of A, B, C	Pass
K5	Names of components and HALs agree with AOSP terminology	Comparison with AOSP documentation	Pass

8.3 Scenario walkthrough result

The combined scenario in Section 7.4 was traced on the diagram from the navigation app to the speakers. Every step uses only components and interfaces that exist in Figure 1, and the scenario shows that Car audio service is the single point where music and navigation audio are arbitrated. **Result:** the architecture supports the scenario without any additional component.

8.4 Results summary

All five requirements are met and all five consistency checks pass. The delivered result is one annotated architecture diagram, one interface table with 13 entries and three traced data flows with 12 steps.

9. Discussion

9.1 What the design shows

The layered structure separates what the driver sees (apps) from how the system works (services and Car layer) and from the hardware (HAL and devices). This has three practical benefits:

- **Portability.** The same apps can run on different hardware because only the HAL and drivers change.
- **Safety and control.** Vehicle data and audio are accessed through controlled services, so permissions and driving restrictions can be enforced in one place.
- **Maintainability.** Each layer can be updated or tested separately.

The diagram also shows a difference between flows: user input (Flow A) mainly travels **down** the stack, while sensor data (Flow B) mainly travels **up**. Projection (Flow C) is different again, because the main content comes from an external device.

9.2 Simplifications and differences from a real system

To keep the diagram readable, some details were deliberately left out or combined. A reviewer should be aware of them:

- **Native audio server and audio policy.** In a real Android system, audio data passes through the native audio server (AudioFlinger) and the audio policy before reaching the Audio HAL. These are folded into interface I7.
- **Origin of the focus request.** The audio focus request is normally made by the app's player through AudioManager and decided by Car audio service. The diagram groups this under Media service for clarity.
- **Audio control HAL.** Automotive Android also has an audio control interface for focus and volume events. It is not drawn separately.
- **Media apps in Android Automotive.** A media app is mainly a service that provides content and playback control, while the system provides the media user interface. The diagram shows the app as one block.
- **Projection audio.** Audio from the phone also uses the car audio path and audio focus. This connection is not drawn to avoid crossing lines, but the audio flow A3 to A5 applies to it.

- **Vehicle networks.** The head unit usually reaches the CAN bus through a gateway or microcontroller, shown as one hardware block.
- **Projection integration.** Manufacturers integrate Android Auto and CarPlay in different ways, so Projection service is a generic model.

9.3 Safety and quality considerations

- **Driver distraction.** The projection and navigation UIs should limit complex input while the vehicle is moving, using the driving state obtained through Vehicle HAL.
- **Latency.** Touch response and audio start-up should be fast. Rear-view camera and warning sounds normally need a much faster start-up than the full infotainment system.
- **Security.** Apps and the phone connection must not have unrestricted access to vehicle networks. The service and HAL layers, with permission checks, provide a control point.
- **Updates.** Over-the-air updates should be able to change apps and services without changing safety-critical ECUs.

10. Conclusion & Future Scope

10.1 Conclusion

A conceptual architecture for an IVI system integrating media, navigation and projection was designed as a six-layer model. Thirteen interfaces were annotated with their protocols and data, and three data flows with twelve steps were traced from input to output. The design meets all requirements of Option 1, and a combined scenario showed that audio focus in Car audio service allows navigation prompts and music to coexist. The diagram is a clear base for understanding how an infotainment stack works and for the further modules of this course.

10.2 Future scope

- Implement a simple Android Automotive media or navigation app and test it on the Android Automotive emulator or a Raspberry Pi setup (Option 2).
- Simulate the message flow between Media service and Audio HAL using Android logs (Option 3).
- Model use cases such as an incoming call interrupting navigation and automatic media resume in UML (Option 4).
- Extend the architecture with Bluetooth telephony, rear-view camera, instrument cluster, multi-zone audio, OTA update and cybersecurity blocks.
- Measure latency and start-up time on real hardware to check the design against timing requirements.

11. References

1. Android Open Source Project, "Android Automotive OS," source.android.com/docs/automotive.
2. Android Open Source Project, "Vehicle HAL," source.android.com/docs/automotive/vhal.
3. Android Open Source Project, "Audio in Android Automotive," source.android.com/docs/automotive/audio.
4. Android Developers, "Android for Cars," developer.android.com/cars.
5. Google, "Android Auto," android.com/auto.
6. Apple Inc., "CarPlay," apple.com/ios/carplay.
7. Module 6 task sheet: IVI In-Vehicle Infotainment Systems (course material).